



16.2. Gated Recurrent Unit (GRU)

The memory of an LSTM, with less baggage.

Previous Sections *(highly recommended reading them first)*

 [16. Recurrent Neural Networks](#)

 [16.1. Long Short-Term Memory Networks \(LSTM\)](#)

Contents:

[1. What is a Gated Recurrent Unit?](#)

[2. Implementing a GRU Model](#)

[2.1. Preparing the Input](#)

[2.2. Building the GRU Model](#)

[2.3. Training the Model](#)

[2.4. Visualize the Results](#)

[2.5. Evaluating the Model](#)

[2.6. Comparing RNN, LSTM, and GRU](#)

[Summary](#)

[References](#)

In the previous section, we introduced Long Short-Term Memory (LSTM), a powerful recurrent architecture designed to overcome the memory limitations of traditional RNNs.

LSTM achieved a significant improvement in forecasting performance because it could preserve important information across longer sequences. However, this

increased capability comes at a cost.

LSTM networks contain multiple gates and a separate memory cell, making them more computationally expensive and slower to train.

This naturally raises an interesting question: ***Can we achieve similar performance with a simpler architecture?*** The answer is yes.

Researchers proposed a lighter alternative known as the **Gated Recurrent Unit (GRU)**. GRUs retain the core idea of gated memory while simplifying the internal structure of the network. In many practical applications, they achieve performance comparable to LSTM while requiring fewer parameters and less training time.

In this section, we will explore how GRUs work and compare their performance against the Baseline model, SimpleRNN, and LSTM using the same Store Sales dataset.

1. What is a Gated Recurrent Unit?

A **Gated Recurrent Unit (GRU)** is a recurrent neural network architecture introduced as a simplified alternative to LSTM.

Like LSTM, GRU was designed to address the vanishing gradient problem and improve the network's ability to learn long-term dependencies.

However, GRU achieves this using only two gates: Update Gate and Reset Gate. Unlike LSTM, GRU does not maintain a separate cell state. Instead, it combines memory and hidden state into a single representation.

As a result, fewer parameters must be learned. Training is typically faster. Memory consumption is lower. Performance often remains competitive with LSTM.

- **Update Gate:** The update gate determines how much information from previous timesteps should be retained. Conceptually, it answers the question: How much of my existing memory should I keep?
 - If the gate value is high, the network preserves historical information.
 - If the gate value is low, the network relies more heavily on newly observed data.

- **Reset Gate:** The reset gate determines how much historical information should be ignored when processing new observations. Conceptually, it answers: ***How much of the past should I forget right now?*** This allows the model to discard irrelevant information when circumstances change.
 - For example, if a store experiences an unusual promotion period, the GRU can learn to temporarily reduce reliance on older sales patterns.

GRU vs LSTM

Although their internal structures differ, both architectures pursue the same objective: learning long-term dependencies within sequential data.

Characteristic	LSTM	GRU
Gates	3	2
Cell State	Yes	No
Parameters	More	Fewer
Training Speed	Slower	Faster
Memory Usage	Higher	Lower
Prediction Accuracy	Often Slightly Better	Often Comparable

There is no universal winner. Many practitioners begin with GRU because it is simpler and faster, then experiment with LSTM if additional performance is required.

2. Implementing a GRU Model

Fortunately, no changes are required to the preprocessing pipeline. Everything prepared for the RNN and LSTM experiments can be reused:

- Train / Validation / Test splits
- Scaling
- Time-based features
- Lag features
- Rolling statistics
- Sliding windows

Only the neural network architecture changes.

2.1. Preparing the Input

As with LSTM, GRU expects three-dimensional input: `(samples, timesteps, features)`

```
train_dataset_gru = train_dataset
val_dataset_gru = val_dataset
test_dataset_gru = test_dataset
```

The sliding-window data was defined during the Recurrent Neural Networks section

2.2. Building the GRU Model

The architecture closely resembles the LSTM model from the previous section.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GRU
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.callbacks import EarlyStopping

# Define the GRU model architecture
num_features = len(selected_features)

gru_model = Sequential([
    GRU(64, activation='relu', input_shape=(window_size, num_features), return_sequences=True),
    Dropout(0.2),
    GRU(64, activation='relu'),
    Dropout(0.2),
    Dense(1)
])

# Compile the GRU model
```

```
gru_model.compile(optimizer='adam', loss='mean_squared_error', metrics=['mae'])
gru_model.summary()
```

The model consists of:

1. A GRU layer that processes the sequence.
2. A second GRU layer for deeper temporal representation.
3. Dropout layers to reduce overfitting.
4. A Dense output layer that predicts future sales.

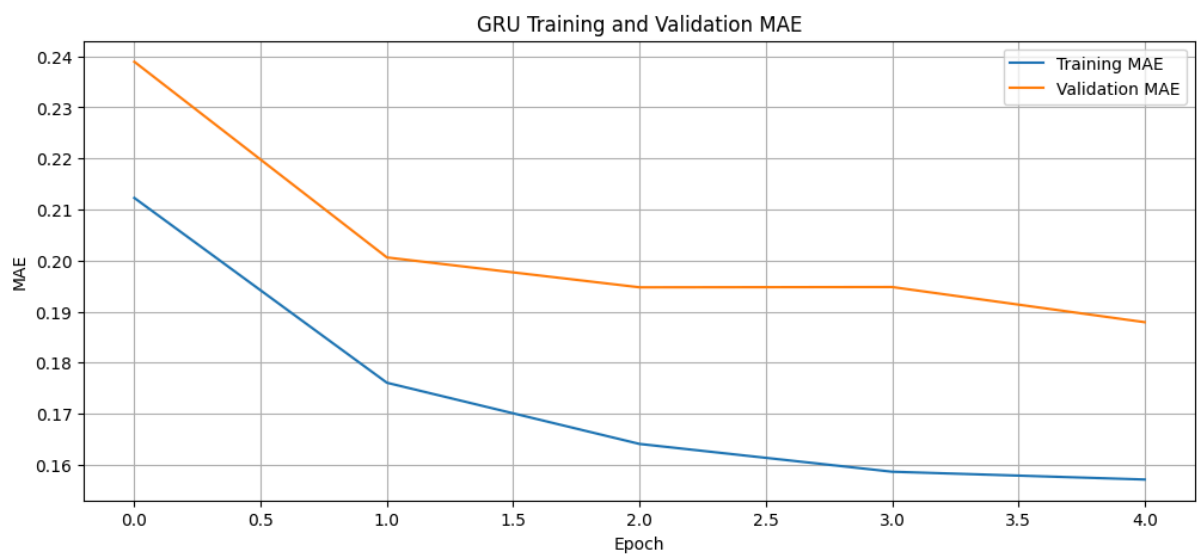
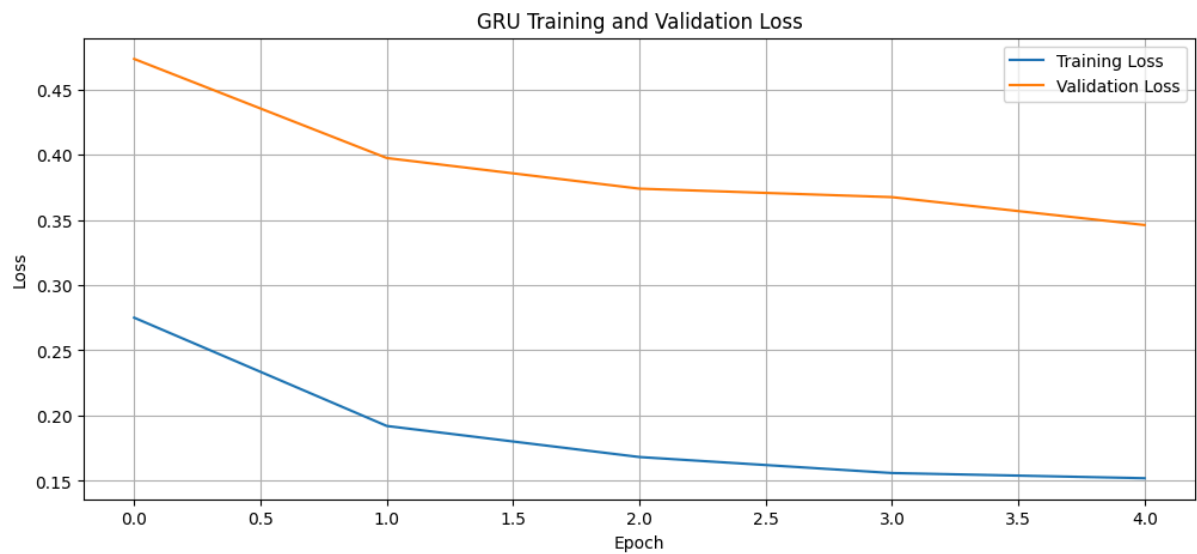
2.3. Training the Model

To prevent overfitting, we again use Early Stopping.

```
early_stopping_gru = EarlyStopping(monitor='val_loss',
                                   patience=2,
                                   restore_best_weights=True)

history_gru = gru_model.fit(train_dataset_gru,
                             epochs=5, batch_size=64,
                             validation_data=val_dataset_gru,
                             callbacks=[early_stopping_gru])
```

2.4. Visualize the Results



2.5. Evaluating the Model

After training, predictions can be generated on the testing dataset.

```

from sklearn.metrics import mean_absolute_error, mean_squared_error

y_pred_scaled_gru = gru_model.predict(test_dataset_gru)

y_test_scaled_gru = np.concatenate([y.numpy() for _, y in test_dataset_gru])
y_test_actual_gru = (main_scaler.inverse_transform(y_test_scaled_gru.reshape(-1, 1)))
y_pred_actual_gru = (main_scaler.inverse_transform(y_pred_scaled_gru))

mae_gru = mean_absolute_error(y_test_actual_gru, y_pred_actual_gru)
mse_gru = mean_squared_error(y_test_actual_gru, y_pred_actual_gru)

rmse_gru = np.sqrt(mse_gru)

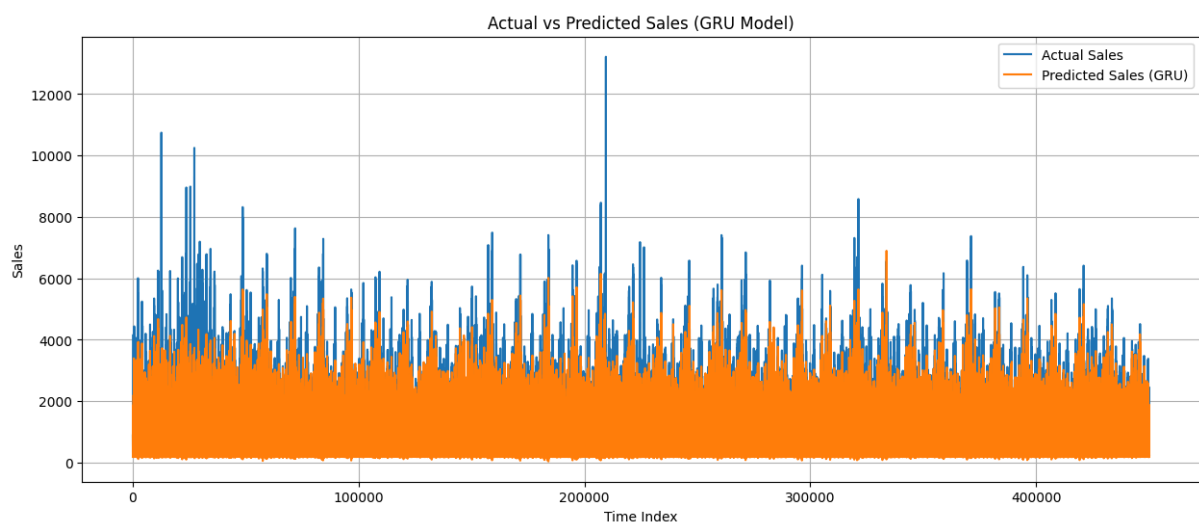
print(f"GRU Test MAE: {mae_gru:.2f}")
print(f"GRU Test MSE: {mse_gru:.2f}")
print(f"GRU Test RMSE: {rmse_gru:.2f}")

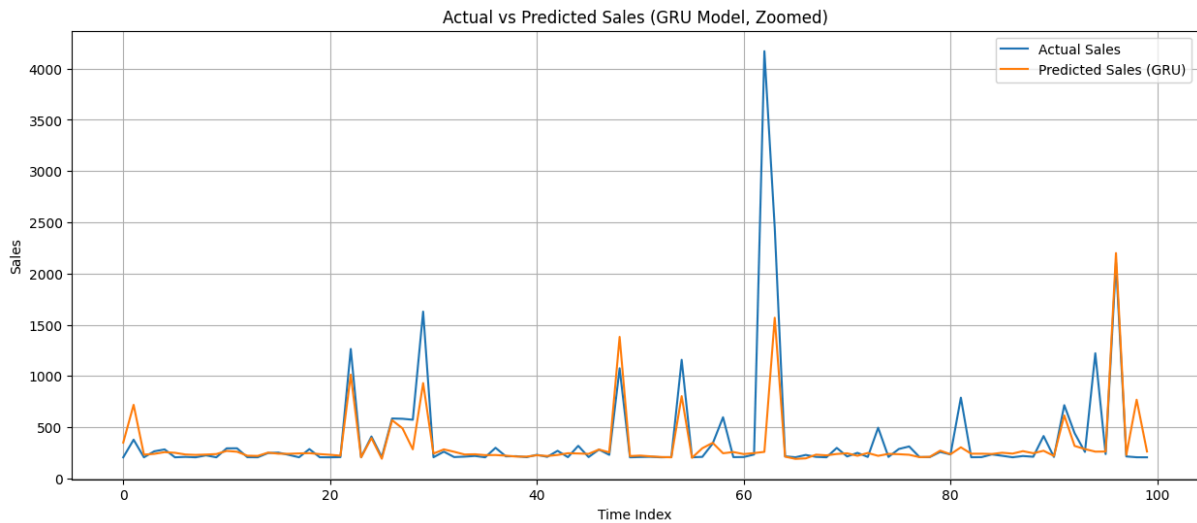
```

GRU Test MAE: 71.98

GRU Test MSE: 36754.51

GRU Test RMSE: 191.71





2.6. Comparing RNN, LSTM, and GRU

To compare the three recurrent architectures fairly (*here we exclude the baseline model*), we examine their final training and validation metrics after training on the same dataset using the same preprocessing pipeline and sliding-window configuration.

Model	Final Training Loss	Final Validation Loss	Final Training MAE	Final Validation MAE
Simple RNN	0.202384	0.454728	0.186625	0.235098
LSTM	0.144782	0.335191	0.151197	0.175430
GRU	0.152001	0.346013	0.157125	0.187927

Several observations can be made from these results.

- The **Simple RNN** achieved the highest training and validation errors among the three models. Although it successfully learned temporal patterns in the sales data, its relatively large validation loss suggests that it struggles to retain useful information across longer sequences. This behavior is expected because traditional RNNs are affected by the vanishing-gradient problem, making long-term dependencies difficult to learn.
- The **LSTM** model produced the best overall performance. It achieved the lowest training loss, lowest validation loss, and lowest validation MAE. The improvement is substantial, indicating that the memory cell and gating mechanisms allow the network to preserve important information over

longer periods of time. For sales forecasting, where weekly cycles, promotions, holidays, and seasonal effects may influence future demand, this ability to retain historical context is particularly valuable.

- The **GRU** model delivered results very close to those of the LSTM. Its validation loss and validation MAE were only slightly higher, suggesting that it captured most of the same temporal relationships despite having a simpler internal structure. Because GRUs contain fewer gates and fewer trainable parameters, they are often faster to train while still maintaining competitive predictive performance.

Overall, the results reveal a clear progression in capability: **Simple RNN** → **GRU** → **LSTM**

As the recurrent architecture becomes more sophisticated, the model becomes better at preserving and utilizing information from earlier time steps. While the difference between LSTM and GRU is relatively small, both architectures provide a noticeable improvement over the traditional Simple RNN. In this experiment, the **LSTM achieved the highest predictive accuracy**, whereas the **GRU offered a strong balance between performance and architectural simplicity**.

Summary

Gated Recurrent Units (GRUs) are a streamlined alternative to Long Short-Term Memory networks that address the vanishing gradient problem while using a simpler architecture. By replacing LSTM's three-gate system and separate memory cell with only an Update Gate and Reset Gate, GRUs reduce computational complexity and often train faster.

In this experiment, the GRU model significantly outperformed both the Previous-Day Baseline and SimpleRNN approaches, demonstrating its ability to capture meaningful temporal patterns in retail sales data.

Although the LSTM still achieved the best overall accuracy, the GRU delivered comparable performance while requiring fewer parameters. This illustrates a common tradeoff in machine learning between model complexity and computational efficiency.

In practice, GRU is often an excellent first choice for sequential forecasting tasks because it provides many of the benefits of LSTM while remaining simpler and faster to train.

References

<https://www.geeksforgeeks.org/machine-learning/gated-recurrent-unit-networks/>

https://d2l.ai/chapter_recurrent-modern/gru.html

Next Section:

 [17. Neural Networks for Texts](#)